

```
C:\Testing\grinder-3.11-binary\grinder-3.11>set CLASSPATH=C:\ProgramData\Grinder\c:\testing\grinder-3.11\bin\grinder.jar
C:\Testing\grinder-3.11-binary\grinder-3.11>set CLASSPATH=C:\ProgramData\Grinder\c:\testing\grinder-3.11\bin\grinder.jar
```

Figure 1: Set the Grinder CLASSPATH

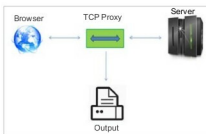


Figure 2: How the TCP proxy works

```
C:\> cd C:\Testing\grinder-3.11-binary\
grinder-3.11
C:\> Testing\grinder-3.11-binary\
grinder-3.11>java net.grinder.Console
```

You will get The Grinder console.

Creating a test

To start with performance testing, we need to have a test. You can create one either by writing your own test from scratch using a script, or you can use The Grinder's proxy tool to record a scenario and to generate a test script. The Grinder comes with a TCP proxy tool. It is a proxy process that can be placed between a server and browser. It records the request and response between a server and browser, and sends the results to a terminal window.

```
C:\Testing\grinder-3.11-binary\grinder-3.11>java net.grinder.TCPProxy -http -con
sole >grinder1.py
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/C:/Users/fujiisu/Downloads/grinder-3.11-binar
y/grinder-3.11\lib\logback-classic-1.0.0.jar!/org/slf4j/impl/StaticLoggerBinder.
class]
SLF4J: Found binding in [jar:file:/C:/Testing/grinder-3.11-binary\grinder-3.11\l
ib\logback-classic-1.0.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple-bindings for an explanation.
13:31:40.472 [main] INFO : Initialising as an HTTP/HTTPS proxy with the paramete
rs:
Request filters: HTTPRequestFilter
Response filters: HTTPResponseFilter
Local address: localhost:8001
13:31:43.991 [main] INFO : Engine initialised, listening on port 8001
```

Figure 3: Output of TCP proxy command



Figure 4: TCP proxy console

It can be used as a very good debugging tool because it lets you see what's going on at the network level. The TCP proxy in The Grinder is used to generate Jython test scripts.

Starting the TCP proxy: To start the TCP proxy, you first need to set up a browser to temporarily use a proxy server. I used Mozilla Firefox as a proxy server.

Setting the proxy: To set the TCP proxy, follow these steps.

1. Open the Firefox menu and click on the *Options* tab.
2. In the *Options* window click on the *Advanced* icon.
3. Select the *Network* tab.
4. In the *Connection* section, click on the *Settings* button.
5. In the *Connection Settings* window, select 'Manual proxy configuration'. Set the HTTP proxy to *localhost* and 8001 as the port. Check the 'Use this proxy server for all protocols' option. Remember that 'No Proxy for the text box' should be empty.
6. Close the *Connection Settings* and *Options* windows by clicking on the *OK* button.

After setting the proxy in the browser, start the TCP proxy. The first step to starting the TCP proxy with an

HTTP filter is as follows:

```
C:\>grinder> java net.grinder.TCPProxy
-http -console >grinder1.py
```

The **grinder1.py* file contains the test script written in Jython.

The output of the TCP proxy is shown in Figure 3.

The console also displays a simple window that allows the TCP proxy to be stopped cleanly. This window allows the user to add comments to the script during recording. The window looks like what's in Figure 4.

The application that you want to test in your browser can now be accessed. I used a simple Web application for creating a new user. In the test scenario, I created a user, logged in, executed a simple SQL query and then logged out using my application.



Note: While running the TCP proxy, make sure that no other tab has been opened. The TCP proxy intercepts every request that the browser makes. So, to avoid unwanted requests, don't open any other tab except the application tab.

Once you're done recording the test, press the *Stop* button in TCP proxy.

Parameterising the test

Let's take a look at a TCP proxy-generated Jython script. I am showing only a part of the script because the whole script is very large.

```
class TestRunner:
    """A TestRunner instance is created for
    each worker thread."""

    # A method for each recorded page.
    def page1(self):
        """POST j_spring_security_check
        (requests 101-102)."""

        # Expecting 302 'Found'
        result = request101.POST('/cdacdm1/
cdacdm1/j_spring_security_check',
        ( NVPair('j_username', 'oracle'),
          NVPair('j_password',
```